

Reviewer Pack — Minimal Rank of Quantum Entanglement Matrix vs BP_ReadTwice ...

Entry e2fc02c175fc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 22:11:10 UTC

Minimal Rank of Quantum Entanglement Matrix vs BP_ReadTwice Circuit Size

Entry ID: e2fc02c175fc **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 22:11:10 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Entanglement Matrices) **Field B** (complexity object): Complexity Theory: BP_ReadTwice Complexity

Statement:

[‘For any n -variable Boolean function f , let M_f be the entanglement matrix of its quantum representation Q_f . Then the minimum rank of M_f is $O(\log \text{size}(Q_f))$ but $\rho(M_{\{IP_2\}}) = \Omega(n)$, where IP_2 is the inner product modulo 2.’, ‘The rank $\rho(M_f)$ of the entanglement matrix M_f for a Boolean function f , when computed over the complex numbers, satisfies $\rho(M_f) \leq \log \text{size}(Q_f)$. For the identity function on n variables, $\rho(M_{\{IP_2\}}) \geq cn$ for some constant $c > 0$.’]

Rationale (proposer’s reasoning):

[‘Entanglement matrices capture the quantum nature of computations and their ranks could potentially expose non-classical properties that distinguish between quantum computations and classical ones.’, ‘If entanglement matrices can be used to determine BP_ReadTwice complexity, it would provide a new angle for understanding the computational power of quantum computers.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 47682f09a08b7324

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support the conjecture, the mean rank of entanglement matrices for random Boolean functions must be $O(\log \text{size}(Q_f))$. To falsify it, any seed must produce an entanglement matrix rank exceeding $\log \text{size}(Q_f)$ by more than 3 standard deviations from the mean.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "entanglement matrix" AND "BP_ReadTwice complexity" - "quantum information theory" AND "minimum rank of entanglement matrices" - "complexity theory" AND "inner product modulo 2"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=244.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def quantum_representation(f):
        n = int(math.log2(len(f)))
        Q_f = [[0] * (2**n) for _ in range(2**n)]
        for i in range(2**n):
            for j in range(2**n):
                if f[i] == f[j]:
                    Q_f[i][j] = 1
        return Q_f

    def entanglement_matrix(Q_f):
        n = int(math.log2(len(Q_f)))
        M_f = [[0] * (2**n) for _ in range(2**n)]
        for i in range(2**n):
            for j in range(2**n):
                if Q_f[i][j] == 1:
                    M_f[i][j] = 1
        return M_f

    def matrix_rank(M):
        n = len(M)
        rank = 0
```

```

for i in range(n):
    if all(x == 0 for x in M[i]):
        continue
    pivot_row = i
    for j in range(i+1, n):
        if abs(M[j][i]) > abs(M[pivot_row][i]):
            pivot_row = j
    M[i], M[pivot_row] = M[pivot_row], M[i]
    rank += 1
    for j in range(n):
        if j != i:
            factor = M[j][i] / M[i][i]
            for k in range(n):
                M[j][k] -= factor * M[i][k]

return rank

def BP_ReadTwice_complexity(f):
    n = int(math.log2(len(f)))
    complexity = 0
    for i in range(1, 2**n):
        if f[i] != f[0]:
            complexity += 1
    return complexity

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    for _ in range(5): # Ensure at least 30 instances per seed
        f = generate_random_boolean_function(n)
        Q_f = quantum_representation(f)
        M_f = entanglement_matrix(Q_f)
        rank = matrix_rank(M_f)
        size_Q_f = len(Q_f)
        complexity = BP_ReadTwice_complexity(f)
        results.append({
            "n": n,
            "rank": rank,
            "size_Q_f": size_Q_f,
            "complexity": complexity
        })

mean_rank = sum(result["rank"] for result in results) / len(results)
std_dev = math.sqrt(sum((result["rank"] - mean_rank)**2 for result in results) / len(results))

conjecture_holds = all(abs(result["rank"] - mean_rank) <= 3 * std_dev for result in results)
counterexample = "" if conjecture_holds else "Rank exceeds expected by more than 3 standard de

return {
    "metric_name": "Entanglement Matrix Rank",
    "metric_value": mean_rank,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_rank)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Rank exceeds expected by more than 3 standard deviations\"")
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to provide evidence for or against the conjecture. | next: Re-run the test with increased time limits and ensure that it completes successfully to gather the necessary data.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12036
2	propose	ollama_remote	glm4:latest	0	0	11637
3	propose	ollama_remote	glm4:latest	0	0	10504
4	preregistration	ollama_remote	glm4:latest	0	0	5698

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	novelty	ollama_remote	glm4:latest	0	0	4623
6	novelty	ollama_remote	glm4:latest	0	0	5604
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15473
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14215
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14176
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18798
11	judge	ollama_remote	glm4:latest	0	0	59998

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 172763 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/e2fc02c175fc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e2fc02c175fc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e2fc02c175fc.tar.gz` (if generated)